

Architecture Review — 2026-07-27

Systems audit of ABRA. Every number below came from running something on 2026-07-27. Where a document and a measurement disagreed, the measurement won and the document is listed as corrected.

Version reviewed: `aff6240` (CHANGELOG 3.22.0). Fixes landed as `40ad1fe` (CHANGELOG 3.23.0).

1. Executive summary

ABRA collects Pokémon Showdown replays, fits a model of what human players click, and uses that model to play. This review looked at whether the numbers it publishes can be trusted.

The headline finding is that one published result was backwards. A switch in the fitting code, `DROP=`, is supposed to refit the model as if a feature did not exist. It only worked on half the data — the half describing "which move did they click", not the half describing "did they switch something in". The feature it was told to remove therefore survived on every switch row, where it quietly turned into a flag meaning *this row is a switch*, and the model fitted a confident weight to it. Nothing errored. The fit exited successfully and wrote a weight file.

Corrected, the conclusion reverses. Dropping "how often people click this move" makes MAG **worse** at predicting human clicks — 30.9% down to 28.7%, not up to 35.3%. The claim shipped in CHANGELOG 3.22.0 and in commit `baa6425` is withdrawn.

The second finding is that a measurement was reading the wrong file. The audit behind "generated Pokémon sets carry too many same-type attacks" called `loadGames('ots')`, intending the open-team-sheet corpus. That function takes an options object, not a name, so the string was ignored and it silently read the ordinary ladder store — where team sheets are optional and 95% of games do not have one. Its own stated premise, *all four moves public, no revelation bias*, was false for almost the whole sample.

Rerun against the corpora that actually carry sheets, **the finding survives and gets stronger**: the gap is +9.9 and +9.4 points on two independent collections of 12,619 and 25,284 sets, where the old figure was +6.2 points from 1,392. The problem was never a shortage of data. Roughly 38,000 human sets were available and 3.6% of them were being used.

The third finding is why nobody caught either one. The test suite lists its tests by hand in a CI file. It named 6 of the 18 test files in the repository, and it never ran `engine/selftest.js` — the file whose own header calls it *"the checks that catch silent wrongness"*. That gate was **failing the entire time**, on 17 files reading the unfiltered ladder store. A failing check that nothing executes is indistinguishable from a passing one.

Fixed and verified. The `DROP` bug, with a guard that refuses to fit if any row escapes and a test that fails on the pre-fix code. The corpus bug, with `loadGames()` now throwing on a bad argument. A derived test runner that discovers 21 checks instead of trusting a typed list. The MEW acceptance gate, which could not be pointed at the artifact it gates and reported "store does not exist" about a 59 MB file that did. The contaminated experiment, quarantined and re-run: **90.8%** of decisive pairs, where the withdrawn figure was 35.4%.

Still broken, deliberately left failing. The site's MAGNEMITE room re-implements the model in the browser and implements 21 of 47 features; the other 26 are silently zero, and the page presents the result as MAG's reasoning. It cannot be fixed without extending the browser bundle, which does not ship the data those features need. 16 files still read the raw ladder store. Five living documents are one release behind the code.

What could not be verified is in section 6 and includes one thing worth naming here: the claim that `merge=union` caused the store to duplicate is supported for three events and **contradicted by a fourth**, which happened 208 commits after the driver was removed.

2. Findings, ranked by blast radius

F1 — `DROP=` applied to half the corpus, inverting a published result

What is wrong. `engine/fit_policy.js` builds a decision's feature vector in two places inside `decisionsFor`: once for a voluntary switch (line 276 pre-fix), once for a move (line 306). The `DROP_IDX` zeroing existed only in the move path.

How it was proved.

```
$ node -e "j=require('./data/policy-weights-nopop.json'); p=j.features.indexOf('priorLogP');
           console.log(j.weights[p], j.standardErrors[p])"
-1.7317684684408094  0.04920161413119673
```

A feature that has been zeroed everywhere has no gradient and cannot acquire a coefficient. This one had -1.73 with a standard error of 0.049 — precisely estimated, and the **opposite sign** to the same feature in the full model ($+0.157$). The mechanism: on switch rows the column still held the real move prior, so it had become a proxy for "this candidate is a switch".

```
$ git show HEAD:engine/fit_policy.js | grep -c 'B\.\featuresFor('
2
$ grep -c 'B\.\featuresFor(' engine/fit_policy.js
1
```

What it could have put in front of a reader. It already did. Held-out top-1 accuracy on the broken file was 35.3% against the full model's 30.9%, and that four-point gain was published as *"Dropping how often people click this makes MAG predict human clicks BETTER"*. Refitting with the drop actually applied gives **28.7%** — the intervention is harmful, not helpful. It also explains the "no-popularity greedy loses to a random bot" result the previous session retracted and blamed on a feature-count straddle: the weights themselves were pathological.

Fixed, and how I know. Both paths route through one `featsFor()`. `assertDropped()` scans the whole corpus and exits non-zero if any row carries a nonzero value in a dropped column. The refit prints:

```
DROPPING priorLogP - refitting everything else around the absence
  drop check priorLogP is identically zero across all 82,836 decisions
priorLogP weight now: 0
```

tests/test-drop-guard.js — 7 checks, passing. It injects one escaped row at index 417 of 600 and requires the guard to catch it (the old duplicate-id check missed 401 duplicates by reading only the first 5,000 lines, so "does it scan the whole thing" is asserted explicitly). Its structural half counts B.featuresFor(call sites and **fails on the pre-fix code**, which is the property that matters.

F2 — the set-sampler audit measured the closed-sheet ladder store

What is wrong. engine/stab_audit.js line 44 read Q.loadGames('ots'). loadGames(opts) does readStore(opts.path); a string has no .path, so readStore(undefined) fell back to STORE = data/games.ladder.jsonl.

How it was proved.

```
$ node -e "Q=require('./engine/quality.js'); g=Q.loadGames('ots');
      console.log(g.length, g.filter(x=>['p1','p2'].some(s=>((x.sheets||{})[s]||[]))
      .some(y=>y&&y.moves&&y.moves.length>=4))).length)"
2245 116
```

2,245 games, 116 with a sheet — 5.2%. Measured directly, the raw stores are: games.ots.jsonl 4,167 games **100%** sheeted; games.bo3.jsonl 2,160 games **99.5%** sheeted; games.ladder.jsonl 17,075 games **1.2%** sheeted. A 5.2% sheet rate is the signature of the ladder store, not of either open-sheet corpus.

What it could have put in front of a reader. It did: the whole of docs/HANDOFF-2026-07-27.md §4, whose premise is "open team sheets (all four moves public, no revelation bias) from 2,245 clean games". The sample was 1,392 sets from 116 games, and the "no revelation bias" guarantee held for 5% of the population it claimed.

Fixed, and how I know. The corpus is named by path and all three are reported separately, because whether a conclusion survives a change of corpus is the question. Every row carries an interval, and the generated sample is matched to the observed one rather than being a quarter of its size.

corpus	clean games	sheeted	human sets	human	generated	gap	95% CI
games.bo3.jsonl (ours, forced)	1,059	99.4%	12,619	23.0%	32.9%	+9.9	[8.8, 11.0]
games.ots.jsonl (external)	2,114	100%	25,284	23.6%	33.0%	+9.4	[8.6, 10.2]
games.ladder.jsonl (what was read)	2,245	5.2%	1,392	28.5%	35.1%	+6.6	[3.1, 10.0]

Two independent collections agreeing at +9.9 and +9.4 is a far better result than one corpus at +6.2. The per-species list is entirely different — Scrafty +47.9, Metagross +40.0, Annihilape +40.0, none of which appeared in the old top rows — and **40 of 58** per-species gaps clear zero. loadGames() now throws a TypeError on a non-object argument; there is no honest default, since guessing which store a caller meant is what caused this.

F3 — the suite ran a third of itself, and the silent-wrongness gate ran never

What is wrong. .github/workflows/tests.yml named each test in its own step: 6 of 18 files in tests/, and none of engine/selftest.js, engine/conformance.js, engine/validate_selfplay.js.

How it was proved. Running the omitted gate directly:

```
$ node engine/selftest.js
24 passed, 1 failed
FAIL every raw reader of the ladder store declares why
17 file(s) read the ladder store with neither a clean filter nor a RAW-STORE-OK declaration
```

What it could have put in front of a reader. Every behavioural statistic in the project. GARBODOR is the rule that unfiltered ladder games are ~13% usable and must never be a baseline; the check that enforces it was red and unobserved. F4 below is one concrete instance that was live in a gate.

Fixed, and how I know. `tests/run-all.js` derives the list from the filesystem — 21 checks discovered where CI ran 6. It separates *passed*, *failed* and **never ran**; a skip prints its reason and is not counted as a pass. Exit code 2 means "could not run", so a gate whose corpus is gitignored stays listed instead of forgotten. It also warns about `engine/*.js` files that print their own pass/fail summary and are run by nothing — which is how F4 was found.

F4 — the MEW acceptance gate could not be aimed at what it gates

What is wrong. Three defects in `engine/validate_selfplay.js`, the file whose header calls it "the acceptance bar for MEW output".

1. `STORE` was hardcoded to `data/games.selfplay.jsonl` and `argv` was ignored, while `mew_farm.js` ends every run printing *"VALIDATE BEFORE USE: node engine/validate_selfplay.js"*.
2. It baselined "format realism" against `games.ladder.raw-logs.jsonl` **unfiltered** — a GARBODOR violation inside a gate. Comparing a corpus of bots against a corpus that is mostly other people's bots and calling the difference realism is circular.
3. A side-balance check fired on **8 games**. Its guard was `if (!N) continue`, which skips only an empty sample.

How it was proved. Pointed at the real 59 MB run:

```
$ node engine/validate_selfplay.js data/h2h-nopop-greedy.jsonl
FAIL self-play store exists          (x3)
FAIL side balance, mirror: p1 won 87.50% of 8 (95% CI [52.91, 97.76])
SELF-PLAY VALIDATION: 3 passed, 3 failed
```

Three checks failed claiming a 59 MB file was absent. And the n=8 check announced a harness bias while the dedicated 300-battle mirror check **in the same run** reported 53.3% [47.7, 58.9] and passed.

What it could have put in front of a reader. A self-play corpus certified as realistic against a bot-infested baseline; or, in the other direction, a reader taught to ignore failures because the gate cries wolf. The unfiltered baseline also published a wrong figure: real-ladder mega rate is **98.3%** on 1,725 clean games, not the 92.9% quoted in that file's own header.

Fixed, and how I know. Store is an argument, raw-log path derived from it. The baseline goes through `quality.js`. The balance check needs 100 games and otherwise reports *inconclusive* — neither a pass nor a failure, because the sample cannot support either.

```
$ node engine/validate_selfplay.js data/h2h-nopop-greedy.jsonl
ladder baseline filtered to 2,245 clean games (engine/quality.js isClean)
real ladder: 1,725 CLEAN games · mega 98.3% · immune 1.98% · failed 2.43%
---- side balance, mirror: only 8 game(s) – too few to judge. Not a pass and not a failure.
SELF-PLAY VALIDATION: 16 passed, 0 failed
```

F5 — the site scores a 21-feature model and calls it MAG

What is wrong. `web/index.html` re-implements `featuresFor` in the browser. It assigns 21 of 47 features; the remaining 26 are never written and hold the zero the array was filled with.

How it was proved.

```
$ node -e "html=fs.readFileSync('web/index.html','utf8'); B=require('./engine/board.js');
a=new Set([...html.matchAll(/x\[MAGIX\.(\\w+)\]/g)].map(m=>m[1]));
console.log(B.FEATURES.length, a.size, B.FEATURES.filter(f=>!a.has(f)).length)"
47 21 27
```

`data/mag.js` was separately stale at 46 features, missing `deadNoLastMove` — regenerated, since it is derived from `board.js` and simply had not been rebuilt.

Why the existing test passed. `tests/test-mag-page.js` compares page features to stored engine features over nine fixture positions. A feature the page never writes reads zero, and a fixture position where the engine also scored zero agrees perfectly. The test also reported only the single largest disagreement, so a page differing on twenty features looked identical to one differing on one.

What it could have put in front of a reader. Everything in the MAGNEMITE room. It is presented as MAG's reasoning and it is a different, weaker model.

Not fixed. Made loud. The 26 features cannot be written into the page as it stands: `data/mag.js` does not ship the fields they need — a bundled move has no accuracy field at all. Closing this means extending the bundle and porting the logic, or deriving the browser scorer from `board.js`, which is the S13-correct answer and is a larger change than this review should make unverified. Two checks were strengthened instead and both now fail deliberately:

```
FAIL the page scores every fixture position exactly as the engine did
-- 7 feature(s) disagree across 9 fixture cases [accuracy, koTarget, dmgFrac,
tgtMayProtect, movesFirst, priority, volatileOnSelf]
FAIL the page implements every feature in the bundle, not a subset
-- the page assigns 21 of 47 features; 27 are never written and therefore silently 0
```

F6 — the raw-store offender count was overstated, and 12 real ones remain

Reported as 17. Measured down to **12**, and the four removed were removed for four different reasons, which is the useful part.

`validate_selfplay.js` was a genuine offender and is fixed (F4). `reprocess.js` was a genuine reader that is **legitimately** so — it rebuilds the store from raw logs, and `isClean` is an analysis filter, not a retention policy; filtering there would silently delete replays that can never be re-fetched. It carries `RAW-STORE-OK` on those merits.

The other three were never reading the store at all. The guard greps for the filename anywhere in a file, so it counted `coach.js` (names it in a `console.log` describing where a finished game goes), `stamp.js` (shows it in a usage docstring as the value a caller passes for `corpus:`), and `mew_farm.js` — which resolves the path **only to refuse to write output over it**, a safety guard counted as a violation of the rule it protects. All three verified as non-readers:

```
$ grep -n "readFileSync\|createReadStream\|open(" engine/{coach,mew_farm,stamp}.js | grep -i ladder
(no output)
```

This matters because the guard is deliberately left failing while offenders remain, so **its number is the project's measure of remaining debt**. Reporting 17 when the truth is 12 makes the one honest signal noisy, and a noisy signal gets ignored.

The tempting fix — strip comments and string literals before matching — is wrong, and worth recording as a near-miss: `readFileSync('data/games.ladder.jsonl')` puts the path inside a string, so stripping strings would create false **negatives**, the one error this check must never make. A separate `RAW-STORE-NOT-READ` declaration is recognised instead, and it has to state what the path is for.

The 12 that remain are real: `calibrate.py`, `chomp-predict.js`, `cores.js`, `ditto.js`, `dynamics.js`, `eval_policy.py`, `flywheel.py`, `jolteon.py`, `playstyle.js`, `pory_baseline.py`, `predictability.py`, `role_atlas.py`. Each is an analysis script that should filter and each needs its own verification that filtering does not move a published number. **Not addressed** — see section 6.

F9 — the sampler's own fix for independence was unreachable

What is wrong. `set_priors.fillSet` filled unrevealed move slots by drawing from $P(\text{move} \mid \text{species})$ independently, which cannot represent a *slot*: Dire Claw and Gunk Shot are both perfectly normal Sneasler moves competing for one place, so marginals paired them at roughly $P(a) \cdot P(b)$.

The part worth naming is that a correction already existed. `sampleMoves()` carries a measured co-occurrence lift built precisely to stop near-substitutes pairing up — and `fillSet` consulted **Smogon's percentages first**, falling through to `sampleMoves` only when Smogon returned nothing, which is rare. For most species the correction was dead code.

How it was proved. `engine/stab_audit.js`, on the corpora that actually carry sheets (F2), and a direct draw of the named case.

What it could have put in front of a reader. Anything computed on generated teams. Only ~1.38 of 4 moves are revealed per set on the ladder, so whatever fills the other 2.6 dominates the result — ADR-001's lesson recurring in a subtler form.

Fixed, and how I know. An open team sheet is the joint distribution, and this project holds **37,903 complete four-move sets across 232 species** in `games.bo3.jsonl` and `games.ots.jsonl` that the sampler was not using. `observedDraw()` takes the sets containing every revealed move — the exact conditional — with maximum-overlap nearest neighbour as the fallback rather than silence, because returning nothing would send the rarest builds back to the sampler that gets them wrong. Marginals are now the third preference, not the first.

corpus	human	before	after	gap before	gap after
bo3 (12,619 sets)	23.0%	32.9%	27.4%	+9.9 [8.8, 11.0]	+4.3 [3.3, 5.4]
ots (25,284 sets)	23.6%	33.0%	27.4%	+9.4 [8.6, 10.2]	+3.7 [3.0, 4.5]
ladder (1,392 sets)	28.5%	35.1%	30.0%	+6.6 [3.1, 10.0]	+1.4 [-1.9, 4.8] — noise

Sneasler holding both Dire Claw and Gunk Shot: **3.3% of 300 draws** → **0.0%**.

`tests/test-set-realism.js` — 6 checks at a 6.0-point threshold, sitting between the pre-fix +9.9 and the current +4.3. Verified in both directions by restoring the old file: **pre-fix 3 passed / 3 failed, current 6 passed / 0 failed**. It asserts direction as well as magnitude, so the sampler cannot be "fixed" into under-producing doubles; and it asserts the observed-set store is populated, because if that store ever empties the sampler reverts to marginals with nothing failing.

Roughly half the gap remains and is not claimed as fixed — species below the 8-set floor still use marginals, and a partially-revealed set still mixes an observed draw with what was on it already.

F7 — five living documents are a release behind

`tests/test-docs-current.js` was in the repository, failing, and unrun:

```
FAIL all living docs are at 3.22.0 — stale: docs/ABRA-whitepaper.md @ 3.21.0,
      docs/ABRA-deck-plain-english.md @ 3.21.0, docs/ABRA-technical-docs.md @ 3.21.0,
      docs/SUMMARY.md @ 3.21.0, docs/MODELS.md @ 3.21.0
```

The 3.22.0 release — the Encore work — broke the project's own same-pass rule. That test's own message gets the remedy right: bumping the header is not the fix, or the version becomes another asserted number.

F8 — untidy, low blast radius

- `champions_sim.js` defaults `SHOWDOWN_PATH` to `/tmp/ps`, a path that exists on no machine here. The error message is excellent, so this costs one confusing line, not a wrong number.
- `tests/test-site-sync.js` guards against stale figures with a **hand-typed list** of old numbers (`/5,199/`, `/7,547/` ...). It is S13 rot inside the test that exists to catch S13 rot: it can only ever catch the specific numbers somebody remembered to add.
- The self-play harness does not stamp the opponent's `--randmove` setting into its records. `paired_h2h.js` prints "SWITCH SETTING NOT RECORDED (run predates the flag)" for a run created minutes earlier, so provenance for that dimension is unrecoverable from the artifact.

3. Changes made

Landed as `40ad1fe`, pushed to `origin/main`.

File	Change	Why
engine/fit_policy.js	Both feature-build paths routed through one <code>featsFor()</code> ; added <code>assertDropped()</code> ; exported it	F1 — the drop reached one of two paths
engine/quality.js	<code>loadGames()</code> throws <code>TypeError</code> on a non-object argument	F2 — a string silently selected the default store
engine/stab_audit.js	Corpus by path, three corpora reported separately, Wilson intervals on every gap, generated sample matched to observed, relative paths, no silent-empty fallback	F2 — wrong corpus, no intervals, unrunnable off this machine
engine/validate_selfplay.js	Store is <code>argv[2]</code> , raw-logs derived from it, baseline filtered through <code>quality.js</code> , balance check needs $n \geq 100$	F4 — three defects
tests/run-all.js	New. Derives the check list; 21 discovered; distinguishes never-ran from passed; exit 2 = skip; warns on unrun engine checks	F3 — hand-typed CI list
.github/workflows/tests.yml	One step calling the runner, replacing 6 hand-named steps	F3
tests/test-drop-guard.js	New. 7 checks; behavioural + structural; fails on pre-fix code	F1 regression guard
tests/test-encore-gate.js	Moved from <code>engine/encore_test.js</code> ; absolute <code>C:/Users/willj/...</code> paths removed	Handoff §6.3; it could never have run on CI
tests/test-mag-page.js	Fixture check reports how many features disagree; new structural coverage check	F5 — a check that passed on a 21-feature page
data/mag.js	Regenerated from <code>board.js</code>	F5 — stale at 46 features
data/policy-weights.json , -nopop.json	Both refitted against the gated <code>deadNoLastMove</code> , nopop refitted again after F1	Handoff §0 and F1
.gitignore	<code>data/quarantine/</code>	<code>git add -A</code> staged 177 MB of withdrawn data; one file exceeds GitHub's 100 MB limit
engine/set_priors.js	<code>observedSets()</code> / <code>observedDraw()</code> ; whole observed sets tried before the marginal paths	F9 — independence, and a correction that was unreachable
tests/test-set-realism.js	New. 6 checks; verified to fail on the pre-fix sampler	F9 regression guard
engine/selftest.js	Recognises <code>RAW-STORE-NOT-READ</code> for files that name the path without opening it	F6 — the count was overstated by 3
engine/coach.js , mew_farm.js , stamp.js	<code>RAW-STORE-NOT-READ</code> declarations	F6 — verified non-readers
engine/reprocess.js	<code>RAW-STORE-OK</code> on its merits	F6 — rebuilding the store must read dirty records
CHANGELOG.md	3.23.0, 3.24.0	Same-pass rule

Test results.

	before	after
Checks CI executed	6 of 18 files	22 discovered, all executed locally
run-all.js	did not exist	18 passed, 3 failed, 1 skipped
engine/selftest.js	24 passed, 1 failed (17 offenders)	24 passed, 1 failed (12 offenders)
tests/test-set-realism.js	did not exist	6 passed ; pre-fix code gives 3 passed / 3 failed
Same-type-attack gap (bo3)	+9.9 points	+4.3 points
engine/validate_selfplay.js on a real run	3 passed, 3 failed	16 passed, 0 failed , 1 inconclusive
tests/test-mag-page.js	7 passed, 2 failed, extent hidden	8 passed, 2 failed, extent reported
tests/test-drop-guard.js	did not exist	7 passed
tests/test-encore-gate.js	unrunnable off this machine	passes from tests/

The three remaining failures are F5, F6 and F7. All three are pre-existing, none is masked, and each is listed above with what closing it requires.

4. Numbers corrected

Every location was checked, not just the changelog. `grep -r1` across `docs/*.md`, `CHANGELOG.md`, `README.md`, `CLAUDE.md`, `web/index.html` and `data/status.js`.

Figure	Was	Is	Appears in	Status
Held-out top-1, popularity dropped	35.2% / 34.6% ("better")	28.7% (worse)	HANDOFF-2026-07-27.md , MAG-VARIABLES.md	Retracted. F1
Held-out top-1, full model	34.6%	30.9%	HANDOFF-2026-07-27.md , MAG-VARIABLES.md	Corrected (gated refit)
priorLogP in the no-pop fit	-1.73, SE 0.05	0 (as intended)	data/policy-weights-nopop.json	Fixed
No-pop greedy vs random	35.4% of decisive pairs	90.8% [89.6, 91.8]	HANDOFF-2026-07-27.md , CHANGELOG.md	Re-run clean
Pairs in that run	4,847 (contaminated)	4,823 (2,814 decisive)	HANDOFF-2026-07-27.md	Re-run clean
Same-type-attack gap	+6.2 points	+9.9 [8.8, 11.0] bo3 / +9.4 [8.6, 10.2] ots	CHANGELOG.md , HANDOFF-2026-07-27.md	Corrected. F2
Ground-truth sample	"2,245 clean games"	1,059 / 2,114 games; 12,619 / 25,284 sets	CHANGELOG.md , HANDOFF-2026-07-27.md	Corrected. F2
Incineroar same-type rate	0.0% human → 22.5% gen (n≈40)	0.3% → 23.7% (n=630)	HANDOFF-2026-07-27.md	Corrected
deadNoLastMove weight	-2.943 (ungated)	-3.434 (gated)	HANDOFF-2026-07-27.md , CHANGELOG.md	Corrected
Real-ladder mega rate	92.9% (raw store)	98.3% (1,725 clean)	CHANGELOG.md , validate_selfplay.js header	Corrected. F4
Browser re-implementation	"still 21-feature"	21 of 47 implemented; bundle was 46	HANDOFF-2026-07-27.md , CHANGELOG.md	Confirmed — see note
Store duplications	"three times" / "the fourth"	at least 5 events across 2 stores	CLAUDE.md , .gitattributes , CHANGELOG.md	See section 5

A correction to this review. I first reported the handoff's "21-feature" claim as wrong, on the grounds that the bundle carried 46 features. Both statements are true about different things: the generated bundle was 46, the page's *implementation* is 21. The handoff was right and my first reading was not. Recorded because a review that hides its own corrections is asking to be trusted on faith.

Two figures in HANDOFF-2026-07-27.md were **verified unchanged**: deadNoLastMove on 5.34% of teams, and the 91.8%/55.5%/81.9% clean h2h cells, which I did not re-run (section 6).

5. Causal claims audit

Claim	Verdict	Evidence		
<code>merge=union</code> is the mechanism that duplicated the store	VERIFIED for 3 of ≥ 5 events	The three large doublings sit at ancestry depths 329, 336, 383. Union was added at depth 307 (1011630) and removed at 417 (b6ea2dc). All three are inside that window with the attribute active — confirmed by <code>`git cat-file -p <c>:.gitattributes \</code>	<code>sed 's/#.*//'</code> <code>\</code>	<code>grep merge=union ,</code> <code>filtering</code> <code>comments, because</code> <code>the current file</code> <code>discusses</code> <code>merge=union` in</code> <code>prose and a naive</code> <code>grep matches that.</code>
Removing the driver stops the duplication	DISPROVEN	<code>009af26</code> "store: dedupe after rebase (137 duplicate lines)" sits at depth 625 with no active union attribute — 208 commits after removal. <code>.gitattributes</code> states "The driver had to go, or the duplication returns on the next divergence." The driver went and a duplication returned.		
The store duplicated "three times" (<code>CLAUDE.md</code> , handoff)	DISPROVEN as a count	Measured line counts across every commit touching the store show doublings to 14,188 (d329), 14,361 (d336) and 16,540 (d383); the 137-line event (d625); and a commit that published o lines (91558e9 , depth 14). <code>CHANGELOG.md:863</code> records a sixth in <code>games.bo3.json1</code> — 1,252 duplicate lines of 2,504. <code>.gitattributes</code> says "the fourth duplication" while listing three magnitudes. No document states a count that matches the history.		
<code>git merge -X ours</code> was the cause	DISPROVEN, already retracted	Correctly retracted in CHANGELOG 3.1.2. Recorded here because the retraction is the model to follow.		
A <code>push-all.bat</code> timer was the unattended publisher	DISPROVEN, already retracted	Retracted in CHANGELOG 2.8.1 in favour of the workspace auto-commit. <code>push-all.bat</code> 's own header still describes the retracted diagnosis and points at a deleted file — noted in <code>CLAUDE.md</code> , still uncorrected in the file.		
The workspace auto-commit is the real collision risk	UNVERIFIED — hypothesis	Consistent with the record: 40+ <code>auto:</code> commits appear in the store's history, several at the anomalous line counts. But no test demonstrates the auto-commit landing mid-rebase and causing a duplication. It is the best available explanation, not a tested one.		
The 46-vs-47 feature straddle	INSUFFICIENT — a second	The straddle is real (timestamps confirm the run spanned the edit). But the nopop		

Claim	Verdict	Evidence
caused the 35.4% no-pop result	cause was larger	weights were <i>independently</i> pathological from F1, and the clean re-run returns 90.8% , not something near 50%. The straddle was diagnosed; the drop bug underneath it was not.
The ingest never creates duplicates; they enter only through git reconciliation	CONSISTENT, not proven	<code>durable-ingest.js</code> does read every existing id before appending, and the current store measures 17,075 lines / 17,075 unique ids / 0 duplicates. That is consistent with the claim but does not exclude an ingest path that has simply not fired.
Independent marginal sampling causes over-produced same-type sets	VERIFIED, strengthened	<code>set_priors.fillSet</code> draws each move independently; the effect replicates at +9.9 and +9.4 points on two corpora with non-overlapping intervals, across 40 of 58 species. Stronger evidence than when it rested on one corpus at +6.2.

On the store guard. The current defence is a custom `merge=jsonl-store` driver that unions by id. It is bound per clone by `build/setup-git.sh` and **is** bound here (`git check-attr merge -- data/games.ladder.jsonl → jsonl-store`). Git cannot let a repository bind it automatically, so on a fresh clone the `.gitattributes` entry names a driver that does not exist. That is a real residual exposure and it covers only the git-reconciliation failure mode — the o-line publication at depth 14 was not a merge at all.

6. What I could not do

- **The 12 remaining raw-store readers (F6).** Not attempted. Each is an analysis script that should filter, and each needs its own verification that filtering does not move a number it has already published. Adding `RAW-STORE-OK` to all 12 would turn the check green while changing nothing, which is worse than leaving it red. The four that were resolved were resolved on their individual merits, not by blanket declaration.
- **Roughly half the set-sampler gap (F9).** +9.9 down to +4.3, not to zero. Species with fewer than 8 observed complete sets still draw from marginals, and a partially-revealed set still mixes an observed draw with what was already on it. Both are tractable; neither is done.
- **The browser re-implementation (F5).** Left failing on purpose. Fixing it means extending `data/mag.js` with fields it does not carry and porting 26 features, unverifiable tonight against a page I cannot exercise in a browser. The audit constraint says leave an unverifiable fix undone.
- **The five stale living documents (F7).** Bumping the version headers would satisfy the check and assert something false. Their content needs to absorb this review's corrections, which is section 4's table applied to five documents.
- **Handoff §6 items 2, 5, 6, 7, 8, 10** — joint scoring wiring, volatiles, the defensive type chart, mega item detection, CHOMP regeneration, hardcoded names. Item 4, the sampler, is done (F9). These touch `board.js`, and each needs a 12-minute refit plus a re-measurement to state honestly, so they belong in one batch with a single refit rather than piecemeal.

On volatiles specifically (§6.5), the scope is larger than the handoff implies, and I measured why. `engine/durable-ingest.js` records no volatiles at all — grepping it for `-start`, `volatile`, `perish` and `taunt` returns nothing. So the sequence is not "add features to `board.js`". It is:

1. `durable-ingest.js` must parse `|-start|` / `|-end|` and store per-slot volatiles;
2. `reprocess.js` must rebuild the store from the archived raw logs to backfill them — 3,849 ladder games predate raw-log archiving and can never carry them;
3. `board.js` gains the tracking and the features;
4. `mag_bot.js` must populate them live from the protocol stream, or they are zero in play;
5. refit.

If someone does step 3 alone, the features are constant zero across the whole fit corpus and the optimiser assigns them arbitrary near-zero weights that look measured. That is exactly the failure shape of F1 and F5 — a feature that is silently absent while appearing present — in a project that has now been bitten by it twice in one week. Doing this in the wrong order does not produce a partial feature; it produces a confident wrong number. Left undone deliberately.

- **The three clean h2h cells** (91.8% greedy-vs-sampling, 55.5% new-vs-old, 81.9% full-vs-random) were **not re-run**. They are quoted from the handoff. They predate F1, and F1 only touched `DROP=` runs, so they are probably unaffected — but "probably unaffected" is not measured, and they should be re-run before being republished.
- **Whether the auto-commit actually wedges a rebase.** Testable by staging a divergence deliberately, and not tested. It remains the leading hypothesis for a defect blamed on two other causes already.
- **CI cannot run the three simulator checks.** They need a built master checkout of pokemon-showdown. `run-all.js` reports them SKIPPED rather than passing. I did not add an upstream master build to CI because a green CI that silently tracks someone else's master is its own failure mode.
- `data/conformance.json` **changed** as a side effect of running `conformance.js`. I did not audit what moved in it; it is a generated report and was committed as regenerated.
- **This PDF was not built with** `build/omnibus.py`. That script requires weasyprint, which installs on this machine but cannot load its native libraries (`WeasyPrint could not import some external libraries` — the GTK/Pango dependency). `build/md_to_pdf.js` exists for exactly this reason and its own header says so: it prints via headless Chrome and needs nothing installed. The PDF was built with it. Worth recording as its own small finding: `requirements.txt` pins numpy and nothing else, so the documented weasyprint toolchain has never been a reproducible dependency of this project.

7. The rules

Each rule names the failure that produced it and the check that enforces it. **A rule with no check is a preference, and is labelled as one.**

#	Rule	Originating failure	Enforcing check
S12	A value lives in one place and is referenced	Site quoted six dataset sizes in six rooms	tests/test-docs-current.js , engine/conformance.js
S13	If a fact can be derived from an artifact, no human types it	data/status.js called PORY "the win" the day it tied a two-feature baseline	Partial. build/build_pdfs.js --check , tests/run-all.js . No check for the general case — preference outside those two
GARBODOR	Never baseline against the raw ladder store	Bot games used as "real games" ~50 times	engine/selftest.js "every raw reader declares why" — currently failing, 16 files
S7	Store shape invariants hold on every store	401 duplicates while sanity_check.py said none, because it read 5,000 of 7,144+ lines	engine/validate_selfplay.js \$2, engine/dedupe_store.py
NEW — R1	A feature the fit is told to drop is identically zero across the whole corpus, or the fit refuses to run	F1	assertDropped() + tests/test-drop-guard.js (behavioural and structural)
NEW — R2	A corpus is named by path. A loader rejects a non-object argument rather than defaulting	F2	loadGames() throws TypeError
NEW — R3	The set of checks is derived from the repository, never typed	F3	tests/run-all.js ; CI calls only the runner
NEW — R4	A gate is aimed at the artifact it gates, by argument	F4	validate_selfplay.js takes argv[2] ; exit 2 = could-not-run
NEW — R5	Never ran, passed, and failed are three outcomes, not two	F3, F4 — a skip and a pass were indistinguishable	run-all.js reports skips with reasons and warns on unrun engine checks
NEW — R6	A check states the extent of a disagreement, not one example	F5 — "disagrees by 1" for 7 disagreeing features	test-mag-page.js counts and lists
NEW — R7	A statistical check declares a minimum sample and reports <i>inconclusive</i> below it	F4 — "the harness favours a side" from 8 games	validate_selfplay.js MIN_BALANCE_N = 100
NEW — R8	A re-implementation must implement every element of what it mirrors, proved structurally, not by fixture	F5 — 26 features silently zero past a passing test	test-mag-page.js coverage check — currently failing, by design
NEW — R9	Withdrawn data is quarantined, never	The contaminated run; and git add -A staging 177 MB of it	.gitignore data/quarantine/ . Preference — no check asserts a retracted figure's corpus still exists

#	Rule	Originating failure	Enforcing check
	deleted and never committed		
NEW — R10	A causal claim carries VERIFIED / UNVERIFIED / DISPROVEN and its evidence	Two root causes propagated across documents before retraction	No check. This is a preference until something greps documents for untested "X caused Y"

Grounding

- **R1, R2 — fail fast, refuse ambiguous input.** Standard defensive-programming practice; the specific form is Hoare's precondition assertion (*An Axiomatic Basis for Computer Programming*, CACM 1969). A silently-defaulting loader is the "insecure direct default" pattern criticised throughout Ousterhout's *A Philosophy of Software Design* (2018), ch. 10 — "define errors out of existence" applies to ambiguity, not to wrong answers.
- **R3, R5 — derived test inventories and visible skips.** Long-standing continuous-integration practice (Fowler, *Continuous Integration*, 2006); test-runner auto-discovery is the default in pytest and Go's toolchain precisely because hand-maintained suites rot. The passed/failed/skipped tri-state is the JUnit XML schema's own model.
- **R7 — declare power before testing.** Textbook statistics; the specific failure is the "significance from a tiny sample" error described in Ioannidis, *Why Most Published Research Findings Are False* (PLoS Medicine, 2005).
- **R8 — structural equivalence over example-based checking.** This is metamorphic testing (Chen et al., *Metamorphic Testing: A Review*, 2018): assert a relation that must hold over all inputs rather than outputs for chosen ones.
- **R4, R6, R9, R10.** No specific literature. R4 and R6 are ordinary engineering hygiene; R9 and R10 are conventions this project needs because of its own history, and I am not going to invent a citation for them.

Appendix — reproducing this review

```
export SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown

node tests/run-all.js # 21 checks; 18 pass, 3 fail
node tests/run-all.js --list # what runs, what skips, and why
node engine/stab_audit.js # three corpora, intervals on every gap
node engine/validate_selfplay.js data/h2h-nopop-greedy.jsonl
node engine/paired_h2h.js data/h2h-nopop-greedy.jsonl # 90.8% of 2,814 decisive pairs

DROP=priorLogP OUT_WEIGHTS=data/policy-weights-nopop.json node engine/fit_policy.js
```

The two contaminated corpora are at `data/quarantine/*.CONTAMINATED-2026-07-27.jsonl`, uncommitted, so the retraction can be checked against the bytes rather than against this note about them.